A Self-Evaluating Agent Pipeline for Single- and Multi-View Image-to-3D Mesh Generation

Anshuman Rai, Anay Sharma
Department of Computer Science KIET Deemed to be University

***Abstract*—Single-image 3D reconstruction is attractive for content creation, but a single photograph does not uniquely determine unseen geometry. This paper describes an implementation-oriented agent pipeline that coordinates pretrained image-to-3D models, image-quality checks, candidate generation, silhouette-based selection, mesh post-processing, and asset export. The implementation accepts one image or two to four named views, preprocesses each image to a centered foreground cut-out, and routes requests through Hunyuan3D-2 shape-generation variants with TripoSR as an optional fallback. Multiple candidates are generated with different random seeds and compared by a silhouette intersection-over-union (IoU) score. The scoring routine searches axis-aligned camera orientations and enforces distinct directions for different input views. The selected mesh is cleaned, normalized, optionally colorized by projecting the input photograph, and exported as GLB, OBJ, and STL with a JSON report and preview. The notebook also provides a Gradio interface and a batch-evaluation harness. Importantly, the inspected notebook contains no successful end-to-end GPU inference output or completed benchmark table; its first configuration cell terminates because no GPU is detected. Accordingly, this paper documents the implemented design and a reproducible evaluation protocol rather than claiming measured reconstruction accuracy or runtime performance. The method's central practical contribution is the integration of model routing, candidate selection, failure recovery, and export into a single inference workflow.**

***Index Terms*—image-to-3D, 3D reconstruction, mesh generation, Hunyuan3D-2, TripoSR, silhouette intersection-over-union, inference pipeline**

I. Introduction

Creating a usable 3D asset from a photograph can reduce the manual effort required in visualization, prototyping, digital content creation, and interactive applications. Recent reconstruction systems use learned priors to infer plausible geometry from a single image or a small set of views. Large Reconstruction Model (LRM) demonstrates feed-forward single-image reconstruction with a transformer-based architecture [1]. TripoSR develops a fast single-image reconstruction system [2], while Hunyuan3D-2 provides pretrained shape-generation models, including single-image and multi-view variants [3]. Despite this progress, the output remains an inference about three-dimensional structure: surfaces not visible in the input cannot be uniquely recovered from a single view.

A practical application therefore needs more than a call to a pretrained model. It must prepare input images, choose an appropriate inference backend, manage compute limits, compare candidate outputs, repair and normalize meshes, and expose the resulting assets in common formats. The notebook studied in this work implements such a workflow as an inference-only agent. It does not train a new generative model. Instead, it orchestrates existing pretrained models and adds explicit input analysis, candidate selection, failure handling, and output packaging.

The system supports two input modes. In single-view mode, one photograph is passed to a single-image model. In multi-view mode, two to four images are assigned to front, left, back, and right labels and routed first to a multi-view model when that backend is available. Candidate meshes are scored against input silhouettes. If the best candidate falls below a configurable acceptance threshold, the agent proceeds to the next available backend, subject to a soft time budget. This is a heuristic selection strategy, not a guarantee of geometric correctness.

The main contributions documented by this implementation are:

- an end-to-end orchestration workflow for single- and multi-view image-to-mesh inference using existing pretrained backends;
- an input-quality analysis and foreground-preprocessing stage with a fallback background-removal method;
- a candidate-selection routine based on normalized silhouette IoU and a search over 24 axis-aligned camera poses;
- mesh cleanup, optional photo-derived vertex color projection, GPU out-of-memory back-off, and export to GLB, OBJ, and STL; and
- a batch-evaluation harness that records per-object backend, silhouette score, face count, elapsed time, and warnings.

These contributions concern system integration and implementation. The notebook's saved execution state does not demonstrate successful GPU inference or a completed empirical evaluation. The paper therefore separates implemented mechanisms from the experiments that remain to be run.

II. Related Work

A. Single-Image Reconstruction

LRM predicts a neural radiance field from one image using a large transformer trained on extensive multi-view data [1].

TripoSR builds on the large reconstruction model direction and targets fast feed-forward mesh reconstruction from a single image [2]. Such systems use learned shape priors to infer geometry that is not directly observed. This makes them useful for rapid asset generation, but also means that visual plausibility should not be confused with metric accuracy of the complete object.

B. Diffusion-Based 3D Asset Generation

Hunyuan3D-2 describes a family of large-scale models for 3D shape generation and textured asset synthesis [3]. The inspected notebook calls the shape-generation pipeline and selects the mini single-image model or multi-view model according to the number of input views. A turbo configuration is requested when available, with a reduced inference-step count in the implementation. The notebook does not train or fine-tune these models; all model weights are obtained from their pretrained repositories at runtime.

C. Positioning of the Present Work

Unlike a new model architecture, the present system focuses on coordinating available reconstruction backends and operational stages around them. Its silhouette score is used as a lightweight selection heuristic. It should not be interpreted as a substitute for reference-mesh metrics, perceptual evaluation, or human assessment. The contribution is thus a reproducible engineering pipeline and evaluation scaffold, with empirical effectiveness still to be established.

III. System Overview

Fig. 1 summarizes the processing stages implemented in the notebook. The pipeline starts with uploaded or repository sample images. It preprocesses and analyzes each view, determines a backend chain, generates candidate meshes, and evaluates their rendered silhouettes. The best-scoring candidate is then oriented, cleaned, optionally colorized, and exported.

A. Input and Configuration

The notebook is designed for a Google Colab runtime with CUDA GPU access and suggests a T4-class GPU. The main configuration sets a 512-pixel square preprocessed image, an octree resolution of 320, a marching-cubes chunk setting of 8000, a final face-count cap of 80,000, a soft time budget of 900 seconds, and an IoU acceptance threshold of 0.85. Single-view mode requests three seeds for the primary shape backend; multi-view mode requests two. These are configuration defaults in code, not empirically optimized values. If TripoSR is enabled, it is treated as an optional fallback and may be disabled if its CUDA extension cannot be installed.

B. Backend Routing

The backend chain depends on the number of input views and on which imports succeed. When multi-view input and the Hunyuan multi-view backend are available, that backend is attempted first. The Hunyuan mini single-image backend is then appended as a fallback, followed by TripoSR if it is available. Only one backend is intended to remain loaded on the GPU at a time. Loading failures are logged and cause the agent to continue to the next backend. This dynamic chain improves operational flexibility, but it does not imply that every backend is compatible with every environment.

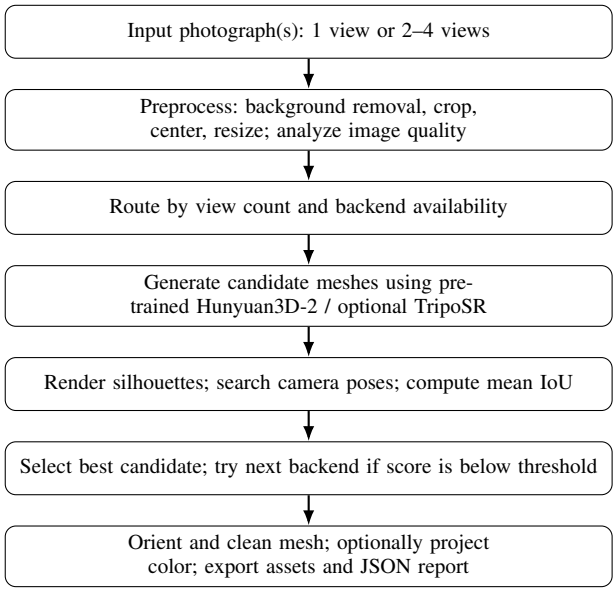


Fig. 1. Implemented inference workflow. The threshold controls routing behavior; it is not a reported performance result.

IV. Methodology

A. Image Preprocessing and Quality Analysis

Images with a sufficiently transparent alpha channel are retained; otherwise, the implementation attempts background removal using `rembg`. If that operation fails, a simple fallback estimates background color from the image corners, thresholds pixel color distance, performs binary morphology, fills holes, and retains the largest connected foreground component. The foreground bounding box is cropped, scaled to fit a 512×512 canvas with an 8% margin, and centered.

For each original image, the analysis stage calculates image dimensions, a Laplacian-variance sharpness proxy, the foreground mask coverage ratio, and the fraction of mask pixels belonging to the largest connected component. Warnings are emitted for low-resolution images, low sharpness, unusually small or large masks, or fragmented masks. These are heuristic quality indicators; no calibration study is included in the notebook to establish universal thresholds for them.

B. View Assignment

Input filenames containing `front`, `left`, `back`, or `right` are used to assign view labels. Remaining images are assigned to the unfilled labels in the order front, left, back, right. If a front view is absent, the first assigned view is promoted to front. This convenience logic assumes that the provided labels or upload order correspond to actual viewpoints. Incorrect labeling can undermine multi-view inference and the evaluation score.

C. Candidate Generation and Resource Recovery

For each backend in the chain, the agent generates one or more candidates using different random seeds. The Hunyuan pipeline is configured for five inference steps in turbo mode or 30 otherwise. The notebook passes the configured octree resolution and chunk size to the generation call. These parameters trade compute and memory demands against mesh detail.

If a CUDA out-of-memory exception occurs, the recovery function frees cached GPU memory, reduces octree resolution by a factor of 0.75 (not below 192), halves the chunk count (not below 2000), and retries up to three times. The modified settings persist for later candidates in the same run. This is a bounded retry mechanism; it cannot guarantee recovery from all GPU failures. A soft time budget is checked between backends, so it is not a strict wall-clock timeout for every individual generation call.

D. Silhouette-Based Candidate Selection

Let T_v denote the binary target silhouette for input view v and $R_{c,p}$ the rendered binary silhouette of candidate mesh c under camera pose p. The intersection-over-union score is

$$\mathrm{IoU}(T_v, R_{c,p}) = \frac{|T_v \cap R_{c,p}|}{|T_v \cup R_{c,p}|}, \tag{1}$$

with score zero when the union is empty. Before comparison, each mask is cropped to its foreground bounding box, padded to a square, and resized to a common resolution. This normalization reduces sensitivity to scale and translation, but also removes absolute size and framing information.

The mesh surface is sampled according to triangle area. The implementation samples 300,000 surface points by default, projects them onto three coordinate planes, and considers both signs of each axis. Each of the six resulting directional silhouettes is evaluated under four in-plane rotations, giving 24 axis-aligned pose candidates. For each input view, the best rotation is chosen for each direction. The multi-view assignment then searches permutations so different input views map to distinct directions. The candidate score is the mean IoU across input views:

$$S(c) = \frac{1}{|V|} \sum_{v \in V} \max_{p \in P_v} \mathrm{IoU}(T_v, R_{c,p}), \tag{2}$$

subject to the distinct-direction assignment used by the implementation. The mesh with the highest mean score is selected. If the current best score is below the configured threshold, the agent may try another backend.

The score has important limitations. Rendering uses an orthographic-style point projection while input photographs may have perspective distortion. Searching over many poses and selecting the best match can inflate the score. More fundamentally, silhouette overlap measures projected outline agreement, not hidden-surface correctness, topology, texture quality, or full 3D geometric distance. A high score therefore does not establish high overall 3D accuracy.

TABLE I
PRINCIPAL IMPLEMENTATION SETTINGS IN THE INSPECTED NOTEBOOK

Setting	Configured value
Preprocessed image size	512×512
Foreground margin	8%
Octree resolution	320
Chunk count	8000
Maximum output faces	80,000
Single-view candidate seeds	3
Multi-view candidate seeds	2
Acceptance threshold	0.85 mean silhouette IoU
Soft time budget	900 s per object
Surface points for scoring	300,000 (default)
Silhouette comparison resolution	128×128 (default)

E. Mesh Post-Processing and Color Projection

The selected mesh is processed to remove degenerate faces and unreferenced vertices where supported, discard small disconnected components, attempt hole filling and normal repair, and decimate meshes above the configured face cap. The vertices are then centered and scaled so the largest bounding-box dimension is one. The mesh is oriented using the recovered front-view assignment, and an additional normal-direction check may invert face winding.

When colorization is enabled, the input photograph is projected onto the mesh using camera-facing visibility logic. The implementation reuses existing vertex colors when available and estimates colors for vertices not directly observed from the input view. This is a simple photo-projection step, not a learned texture synthesis model; hidden surfaces cannot be faithfully colored from a single photograph alone. The system writes GLB, OBJ, and STL files, a preview image, and a JSON report containing the chosen backend, seed, input views, IoU scores, mesh size, warnings, and elapsed time.

V. IMPLEMENTATION

The notebook is organized as sequential Colab cells: GPU/configuration checks, optional Google Drive model caching, dependency installation and repository cloning, core image and mesh utilities, backend and agent logic, an interactive run cell, an optional Gradio interface, batch evaluation, and output bundling. The main implementation uses Python with PyTorch for inference, PIL and NumPy for image handling, SciPy for image morphology and spatial queries, Trimesh for mesh processing and export, and Gradio for the optional interface. It attempts to install the Hunyuan3D-2 and TripoSR repositories and their supporting packages at runtime.

The user-facing Gradio interface accepts a required front image and optional left, back, and right views. It returns a 3D preview, a rendered preview sheet, a textual summary, and downloadable assets. The batch evaluator expects images in a configured directory, either as individual single-view images or as per-object folders with up to four views. For each object, it records mode, selected backend, silhouette IoU, face count, elapsed time, and warnings, then writes a CSV file.

VI. Evaluation Protocol and Current Evidence

A. What Is Currently Verified

The saved notebook contains source code for preprocessing, backend routing, candidate scoring, mesh cleanup, color projection, and batch evaluation. Its first configuration cell reports `RuntimeError: No GPU found`, so the recorded execution does not proceed to model loading or end-to-end inference. The notebook's introductory note states that non-GPU logic was tested on synthetic data, but the file does not include detailed test cases, test logs, or numerical summaries that would support quantitative claims about those tests. No real input images, generated meshes, completed evaluation CSV, per-object metrics, or successful runtime measurements are embedded in the inspected notebook.

Consequently, this manuscript does not report an achieved IoU, average generation time, success rate, or comparison against a baseline. The threshold of 0.85 is only a configured stopping criterion. Model-level performance figures reported in the referenced papers belong to those papers' own experiments and must not be attributed to this implementation.

B. Recommended Experimental Procedure

Before submission as an empirical conference paper, the following evaluation should be run on a documented GPU environment:

1) Assemble a test set of objects with consistent single-view images and, where available, front/left/back/right views. Record image provenance and licensing.
2) Record the exact GPU model, GPU memory, CUDA and PyTorch versions, model checkpoint identifiers, configuration, and software revisions.
3) Run every test object with a fixed configuration and preserve the generated GLB/OBJ/STL files and JSON reports. Repeat stochastic runs where practical.
4) Report sample count, successful-run rate, mean and standard deviation of elapsed time, face count, and silhouette IoU separately for single-view and multi-view modes.
5) Compare against a defined baseline, such as running one backend without candidate selection, and perform an ablation that disables the fallback chain or multi-seed selection.
6) If reference 3D meshes are available, add geometric metrics such as Chamfer distance or F-score under a documented alignment and sampling protocol. Do not infer those metrics from silhouette IoU.
7) Inspect representative successes and failures, including background-removal errors, incorrect view labels, thin structures, hidden surfaces, and GPU-memory failures.

The current batch-evaluation cell can supply timing, face-count, and silhouette-score records once a test directory is populated and inference completes. Its outputs should be treated as measurements only after a successful run; a failed run should be counted and described rather than silently excluded. For multi-view evaluation, the input views should correspond to distinct physical viewpoints. The notebook's assignment procedure can enforce different predicted directions, but it cannot verify whether the supplied images were actually captured from those directions.

C. Metrics and Reporting Requirements

A useful evaluation should distinguish operational reliability from reconstruction quality. Operational measurements include the fraction of objects for which the full pipeline returns an exportable mesh, total elapsed time, number of backend attempts, number of candidates generated, peak GPU memory where available, and the frequency of recoverable or terminal errors. These measures evaluate whether the agent completes its task, not whether its geometry is correct. The JSON report currently records selected-backend identity, the chosen seed, face and vertex counts, per-view silhouette IoU, warnings, and total elapsed time. It does not currently record peak GPU memory, all failed backend attempts in a structured format, or hardware and software provenance; those fields should be added to support a more complete study.

For silhouette-based analysis, the mean and standard deviation should be reported alongside the number of objects and the distribution of per-object scores. Single-view and multi-view cases should not be pooled without separate reporting because their input information and backend paths differ. A paired comparison is preferable when the same objects can be evaluated in both modes. The acceptance threshold should be fixed before the evaluation and sensitivity to the threshold should be reported separately if it is tuned. Otherwise, selecting a threshold after inspecting test results risks overfitting the evaluation procedure.

If reference meshes are available, geometric evaluation should be performed after an explicit alignment protocol. For example, sampled point clouds from the predicted and reference surfaces can be compared using symmetric Chamfer distance, while an F-score can measure the proportion of points within a stated distance tolerance. The sampling count, distance units, normalization, alignment method, and tolerance must be specified. These metrics are not implemented in the current notebook and should be treated as recommended extensions, not as existing system capabilities. Human evaluation can complement geometric metrics by asking raters to score shape plausibility, input-view fidelity, and texture appearance using a fixed rubric; the number of raters and aggregation procedure should be documented.

D. Proposed Evaluation Matrix

Table II describes an evaluation plan that can be implemented using the existing batch harness plus additional logging. It is a proposed protocol, not a record of completed experiments.

A suitable dataset should include a range of object categories, shapes, surface textures, and background conditions rather than only visually simple objects. Where possible, each object should have a consistent set of views and a reference mesh. The split between development and test objects should

TABLE II
PROPOSED EVALUATION MATRIX (NOT YET EXECUTED IN THE INSPECTED NOTEBOOK)

Condition	Purpose	Measurements
Single-view, default agent	Establish baseline pipeline behavior	Completion rate, time, IoU, face count
Multi-view, default agent	Assess behavior with additional views	Per-view and mean IoU, completion rate, time
Single backend, one seed	Reference for orchestration overhead	Time, score, failure rate
Multiple seeds, same backend	Assess candidate-selection effect	Best/mean IoU, candidate count, added time
Fallback disabled/enabled	Assess fallback contribution	Completion rate, attempts, time, score
Different input quality levels	Characterize preprocessing sensitivity	Warnings, failure types, IoU, human review

be fixed before tuning parameters. If objects are selected from a public repository, the paper should identify the source, subset selection criteria, and applicable license. Repeated images of the same object should not be treated as independent samples when reporting uncertainty.

E. Failure Analysis

Failures should be categorized rather than collapsed into a single score. Relevant categories include foreground extraction removing part of an object, an input mask containing background regions, incorrect view assignment, backend import or checkpoint loading errors, CUDA out-of-memory exceptions, mesh-generation failures, degenerate or fragmented geometry, and poor silhouette alignment. The current code logs many runtime events and returns warnings for input quality and low final IoU, but its evaluation harness converts exceptions into a failed row with zero-valued score, face count, and time. For scientific reporting, a zero score in that row must not be interpreted as an observed IoU of zero for a generated mesh. Failure status should be represented separately from valid metric values, and means should state whether they are computed over successful reconstructions only or include a clearly defined failure penalty.

VII. DISCUSSION

The system addresses several practical issues encountered when turning model checkpoints into an application. First, input analysis makes common preprocessing failures visible to the user. Second, backend routing provides a fallback path when a model cannot load or the candidate score does not meet the configured threshold. Third, candidate selection uses the input silhouette rather than relying only on a single stochastic output. Finally, mesh repair, normalization, color projection, and multiple export formats make the output more useful to downstream tools.

These mechanisms also introduce trade-offs. Trying multiple seeds increases the number of inference calls and may increase latency. The silhouette objective is inexpensive relative to full geometric evaluation, but it can favor meshes with a matching outline and incorrect internal or hidden geometry. Normalizing masks removes translation and scale cues, and orthographic projection is only an approximation for perspective photographs. The simple fallback background remover may fail when the foreground and background have similar colors or when corner pixels contain the object. Photo-based color projection has incomplete evidence for surfaces not visible in the source image.

Reproducibility is another concern. The notebook installs a broad set of packages dynamically and clones the default branch of model repositories. Such behavior can make later runs differ as dependencies, code, or checkpoints change. A publication-ready experiment should pin package versions and repository commits, record checkpoint identifiers, and archive configuration and output reports. Licensing terms for pretrained models must also be reviewed before public release or commercial use; model availability does not automatically imply unrestricted use.

VIII. LIMITATIONS AND FUTURE WORK

The most immediate limitation is the lack of a successful end-to-end GPU run in the inspected artifact. The architecture and evaluation loop are implemented, but the paper cannot yet establish empirical effectiveness. Future work should first complete a controlled evaluation with a representative test set and preserve raw outputs. Additional improvements could include calibrated quality-warning thresholds, confidence estimates that account for viewpoint ambiguity, perspective-aware silhouette rendering, and an independent geometric metric where ground-truth meshes exist. Robust view validation and explicit camera metadata would also reduce ambiguity in multi-view inputs.

The current approach relies on pretrained backends and does not introduce a new learned reconstruction model. Its contribution should therefore be assessed as an integration and candidate-selection system. Any claim that the orchestration improves accuracy or efficiency requires a baseline comparison and an ablation study. Similarly, a claim of superior reconstruction quality requires geometric or perceptual evidence beyond the implemented silhouette score.

IX. CONCLUSION

This paper documents an inference-oriented agent that coordinates pretrained image-to-3D models with preprocessing, image-quality warnings, multi-view routing, multi-seed candidate generation, silhouette-IoU selection, mesh cleanup, optional color projection, and export. Its implementation includes GPU-memory back-off, a Gradio demonstration interface, and a batch-evaluation harness. These components form a practical workflow around existing reconstruction systems rather than a newly trained 3D model. Because the inspected notebook stops at the GPU check and contains no completed benchmark outputs, no quantitative performance conclusion is warranted at this stage. A successful GPU run, documented test set, baseline comparison, and geometric evaluation are required before the work can be presented as an empirically validated conference contribution.

REFERENCES

[1] Y. Hong, K. Zhang, J. Gu, S. Bi, Y. Zhou, D. Liu, F. Liu, K. Sunkavalli, T. Bui, and H. Tan, "LRM: Large Reconstruction Model for Single Image to 3D," arXiv:2311.04400, 2023.

[2] D. Tochilkin, D. Pankratz, Z. Liu, Z. Huang, A. Letts, Y. Li, D. Liang, C. Laforte, V. Jampani, and Y.-P. Cao, "TripoSR: Fast 3D Object Reconstruction from a Single Image," arXiv:2403.02151, 2024.

[3] Z. Zhao *et al.*, "Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation," arXiv:2501.12202, 2025.

[4] M. Liu, C. Xu, H. Jin, L. Chen, M. V. T., Z. Xu, and H. Su, "One-2-3-45: Any Single Image to 3D Mesh in 45 Seconds without Per-Shape Optimization," arXiv:2306.16928, 2023.